Assignment-2

D.Suraj
2303A53006
Batch : 46

Task 1 — Statistical Summary using Google Gemini (Colab) Create a

Python function that finds:

- Mean
- Minimum
- Maximum

from a list of numbers using **Gemini in Colab**.

Commands + Code + Text Run all

Gemini

[1]

```python
import math
def calculate_statistics(numbers):
    """
    Calculates the mean, minimum, and maximum of a list of numbers.

    Args:
        numbers (list): A list of numerical values.

    Returns:
        dict: A dictionary containing 'mean', 'min', and 'max' values.
              Returns None for any statistic if the list is empty.
    """
    if not numbers:
        return {"mean": None, "min": None, "max": None}
    total_sum = sum(numbers)
    count = len(numbers)
    mean = total_sum / count
    minimum = min(numbers)
    maximum = max(numbers)
```

```python
    if not numbers:
        return {"mean": None, "min": None, "max": None}
    total_sum = sum(numbers)
    count = len(numbers)
    mean = total_sum / count
    minimum = min(numbers)
    maximum = max(numbers)

    return {"mean": mean, "min": minimum, "max": maximum}

# Example usage:
my_list = [10, 20, 5, 15, 30, 25]
stats = calculate_statistics(my_list)
print(f"For the list {my_list}:")
print(f"  Mean: {stats['mean']}")
print(f"  Minimum: {stats['min']}")
print(f"  Maximum: {stats['max']}")

empty_list = []
empty_stats = calculate_statistics(empty_list)
print(f"\nFor an empty list:")
```

Terminal

Commands + Code + Text Run all

[1]

```python
print(f"  Statistics: {empty_stats}")

single_number_list = [42]
single_stats = calculate_statistics(single_number_list)
print(f"\nFor a list with a single number {single_number_list}:")
print(f"  Mean: {single_stats['mean']}")
print(f"  Minimum: {single_stats['min']}")
print(f"  Maximum: {single_stats['max']}")
```

```
For the list [10, 20, 5, 15, 30, 25]:
  Mean: 17.5
  Minimum: 5
  Maximum: 30

For an empty list:
  Statistics: {'mean': None, 'min': None, 'max': None}

For a list with a single number [42]:
  Mean: 42.0
  Minimum: 42
  Maximum: 42
```

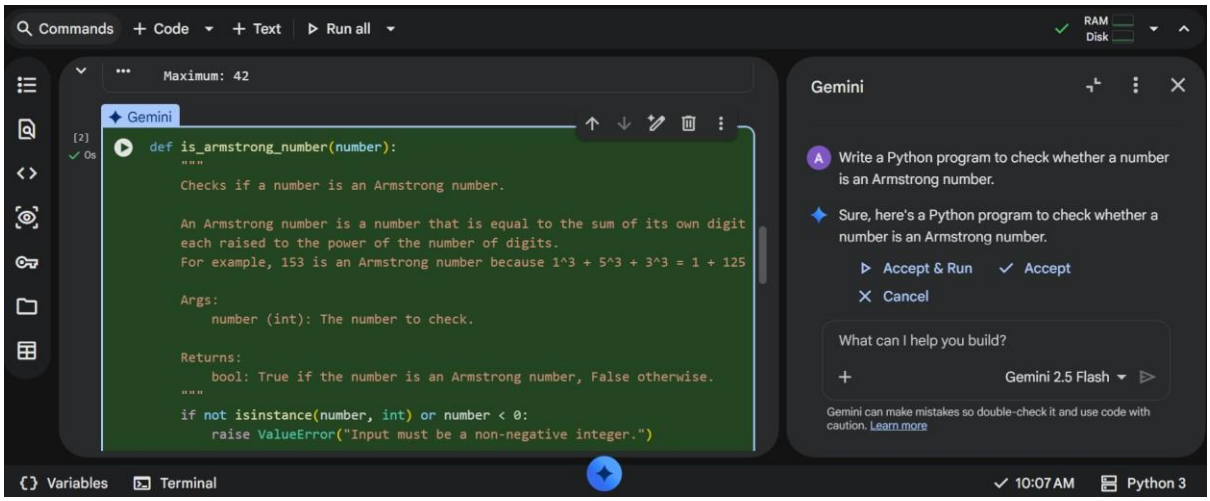
Variables Terminal

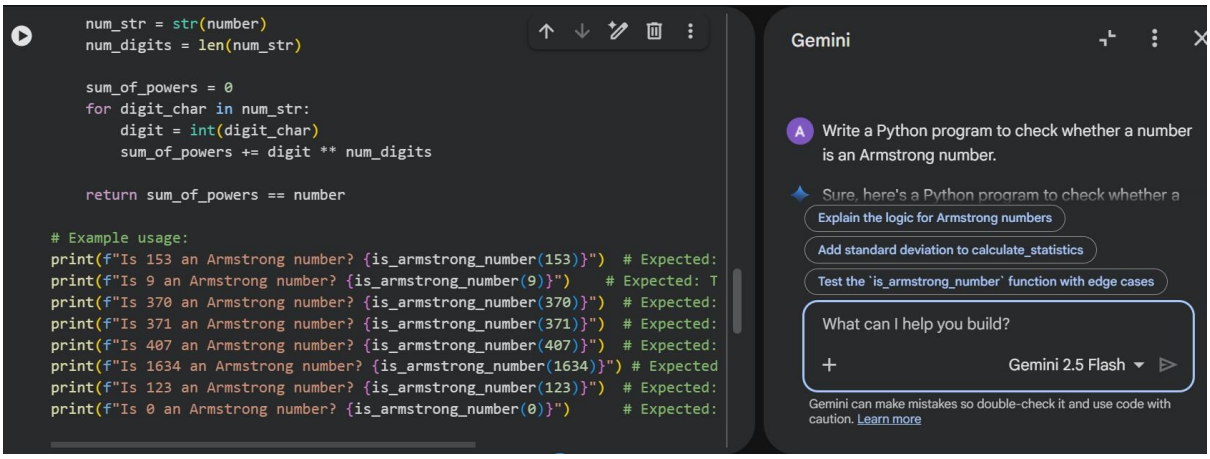
Task 2 — Armstrong Number (Gemini vs Copilot) Generate

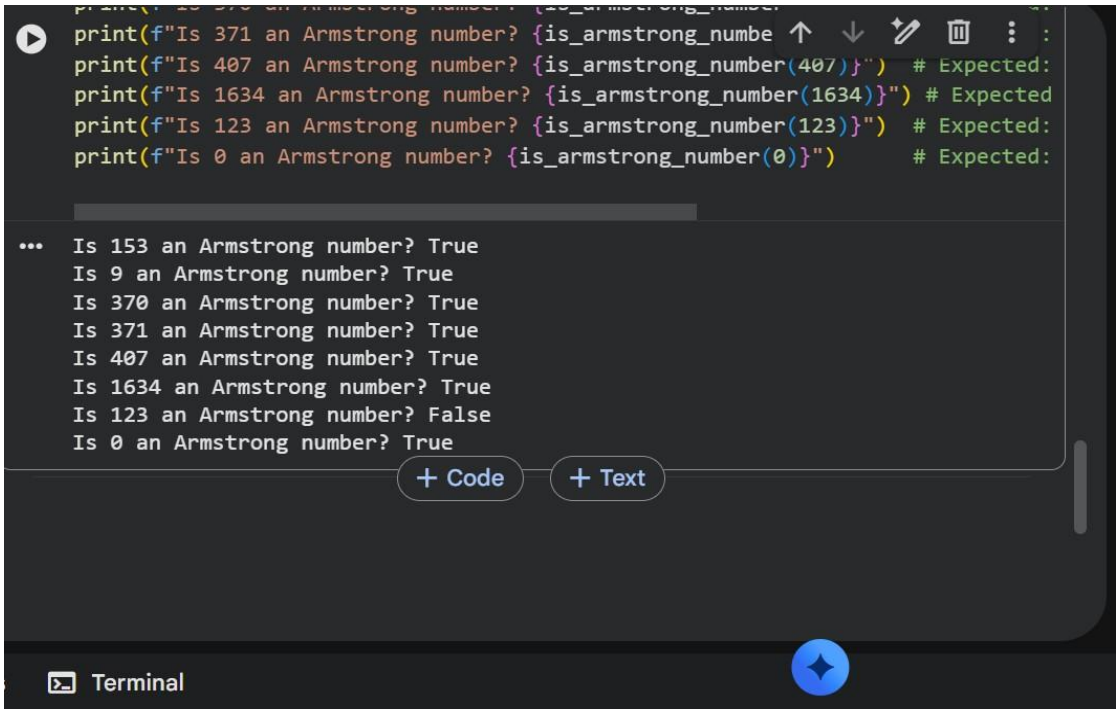
Armstrong number checker using:

- Gemini

- GitHub Copilot ,Then compare them.



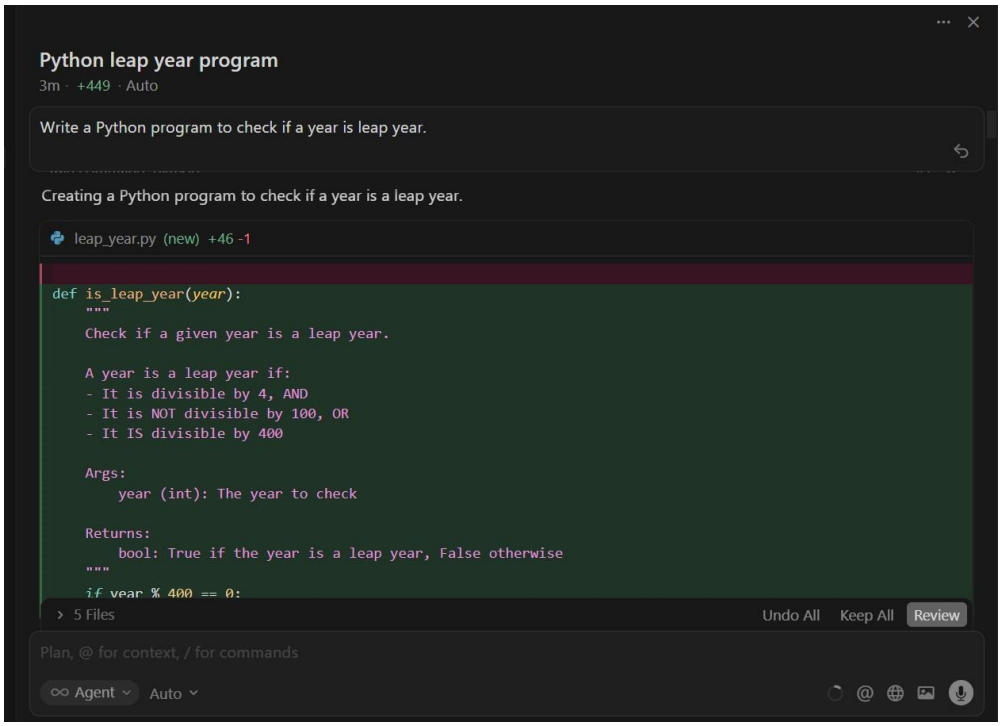




Feature	Gemini	Copilot
Logic	Uses step-by-step digit extraction and power calculation, easy to follow	Uses the same core logic but often written in a more compact style
Code Clarity	Well-structured code with meaningful variable names and comments	Shorter code, fewer comments, more developer-style
Readability	Very beginner-friendly and easy to understand	Slightly less verbose but clean and professional

Task 3 — Leap Year using Cursor AI

Generate leap year program using **two different prompts**



Python leap year program

5m · +449 · Auto

give me optimized code with comparison

OPTIMIZATION_COMPARISON.md (new) +98

```
# Leap Year Implementation: Original vs Optimized

## Overview
This document compares the original and optimized implementations of the leap year checking function.

## Code Comparison

### Original Version (4 if-elif statements)
```python
def is_leap_year(year):
    if year % 400 == 0:
        return True
    elif year % 100 == 0:
        return False
    elif year % 4 == 0:
        return True
    else:
        return False
```
```

5 Files Undo All Keep All Review

Plan, @ for context, / for commands

Agent Auto

Python leap year program

6m · +449 · Auto

give me optimized code with comparison

Benefits of Optimized Version:

- 90% fewer lines of code (10 → 1 line)
- More Pythonic and readable
- Single boolean expression instead of multiple branches
- Same correctness (verified with test cases)
- Same performance (both are O(1))

Run `python comparison.py` to see the detailed comparison with performance benchmarks. Both implementations produce identical results; the optimized version is more concise and Pythonic.

4 Files Undo All Keep All Review

leap_year.py +46 -1

test_leap_year.py +76 -1

leap_year_optimized.py +38 -1

comparison.py +191 -1

Plan, @ for context, / for commands

Agent Auto

Task 4 — Odd/Even Sum (Manual + AI Refactor)

Manual Code

```python
# Tuple of numbers
t = (10, 15, 20, 25, 30, 35)

even_sum = 0
odd_sum = 0

# Loop through each number
for num in t:
    if num % 2 == 0:
        even_sum += num
    else:
        odd_sum += num

print("Sum of Even Numbers:", even_sum)
print("Sum of Odd Numbers:", odd_sum)
```

Output:

```
Sum of Even Numbers: 60
Sum of Odd Numbers: 75
```

AI Refactored Code (Optimized Version)

```python
def calculate_even_odd_sum(numbers):
    """Returns the sum of even and odd numbers from a tuple."""
    even_sum = sum(num for num in numbers if num % 2 == 0)
    odd_sum = sum(num for num in numbers if num % 2 != 0)
    return even_sum, odd_sum


numbers = (10, 15, 20, 25, 30, 35)
even, odd = calculate_even_odd_sum(numbers)

print(f"Sum of Even Numbers: {even}")
print(f"Sum of Odd Numbers: {odd}")
```

Output :

```
Sum of Even Numbers: 60
Sum of Odd Numbers: 75
```